

第六次实验

1. 完成本次实验任务的思路

本次实验的主要任务是利用 Python 实现算术编码算法，并完成对文本文件的压缩处理。算术编码是一种经典的无损数据压缩算法，其核心思想并不是像霍夫曼编码那样为每个字符分配独立编码，而是将整个字符串映射为区间 $[0, 1)$ 内的一个实数，从而实现数据压缩。

实验开始时，首先需要读取输入文本文件中的全部内容，并统计文本中每个字符出现的频率。程序利用 Python 的 Counter 类自动统计字符频率，并根据字符概率逐步划分 $[0, 1)$ 区间。每个字符都会对应一个唯一的概率子区间，字符出现概率越高，其对应区间越大。

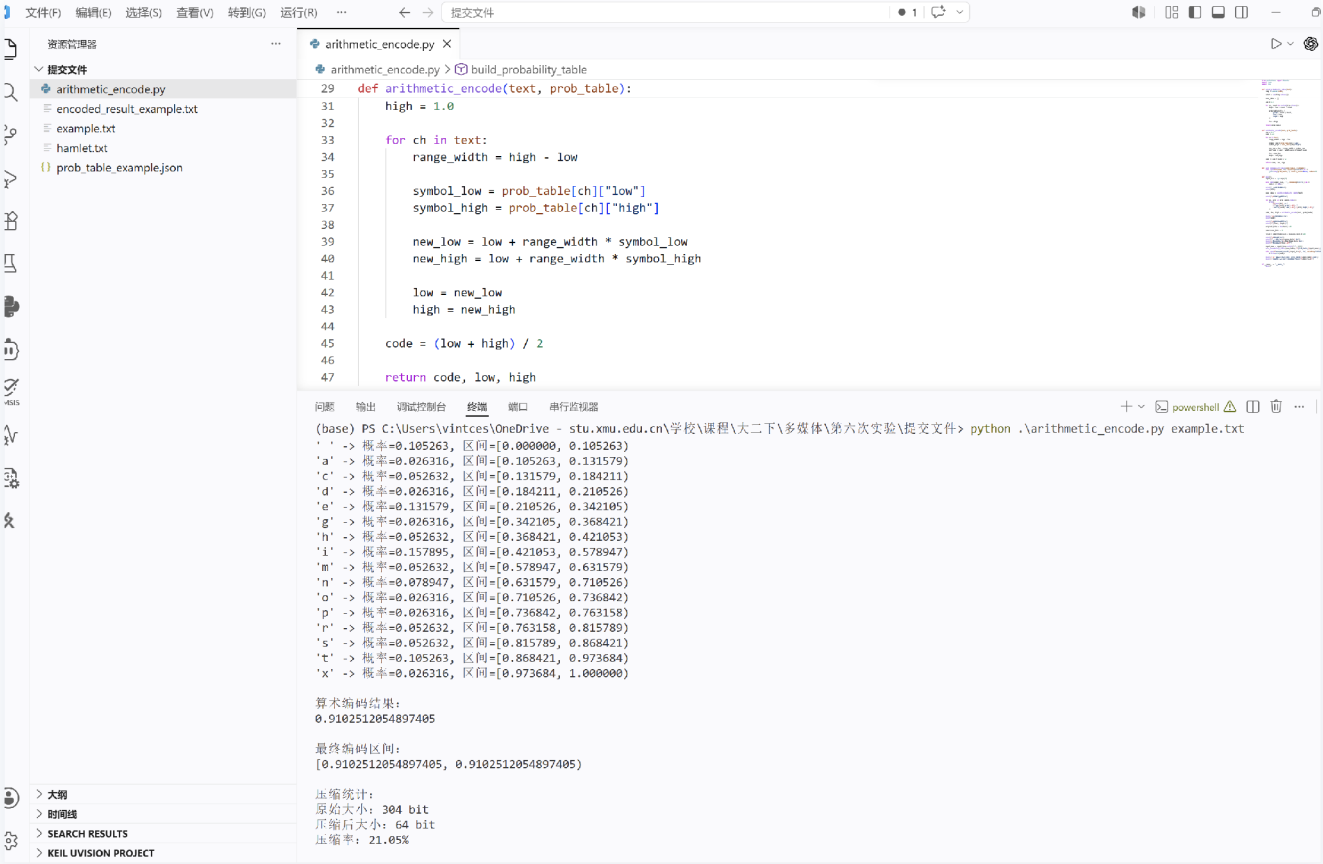
随后程序开始进行算术编码。编码过程中，程序初始区间为 $[0, 1)$ ，然后依次读取文本中的字符，根据当前字符对应的概率区间不断缩小编码范围。随着字符不断输入，最终得到一个非常小的区间，而该区间中的任意一个实数都能够唯一表示整个输入文本。实验中采用最终区间的中点作为最终编码结果输出。

为了验证算术编码算法的正确性与压缩效果，本实验分别对 example.txt 短文本以及 hamlet.txt 长文本进行了编码测试。程序能够自动生成字符概率区间表，并将编码结果保存到文件中，同时统计原始文本大小、压缩后大小以及压缩率等信息，从而分析算术编码对不同规模文本的压缩性能。

整个程序采用 Python 编写，主要利用 collections.Counter 实现字符频率统计，并通过区间迭代不断缩小编码范围，实现完整的算术编码过程。同时程序还支持命令行参数输入文件名，使得同一程序能够方便地对不同文本进行编码实验，提高了程序的通用性与可扩展性。

2. 运行程序截图和简要说明

(1) example.txt 原始文件、概率区间表、编码结果与程序运行结果截图

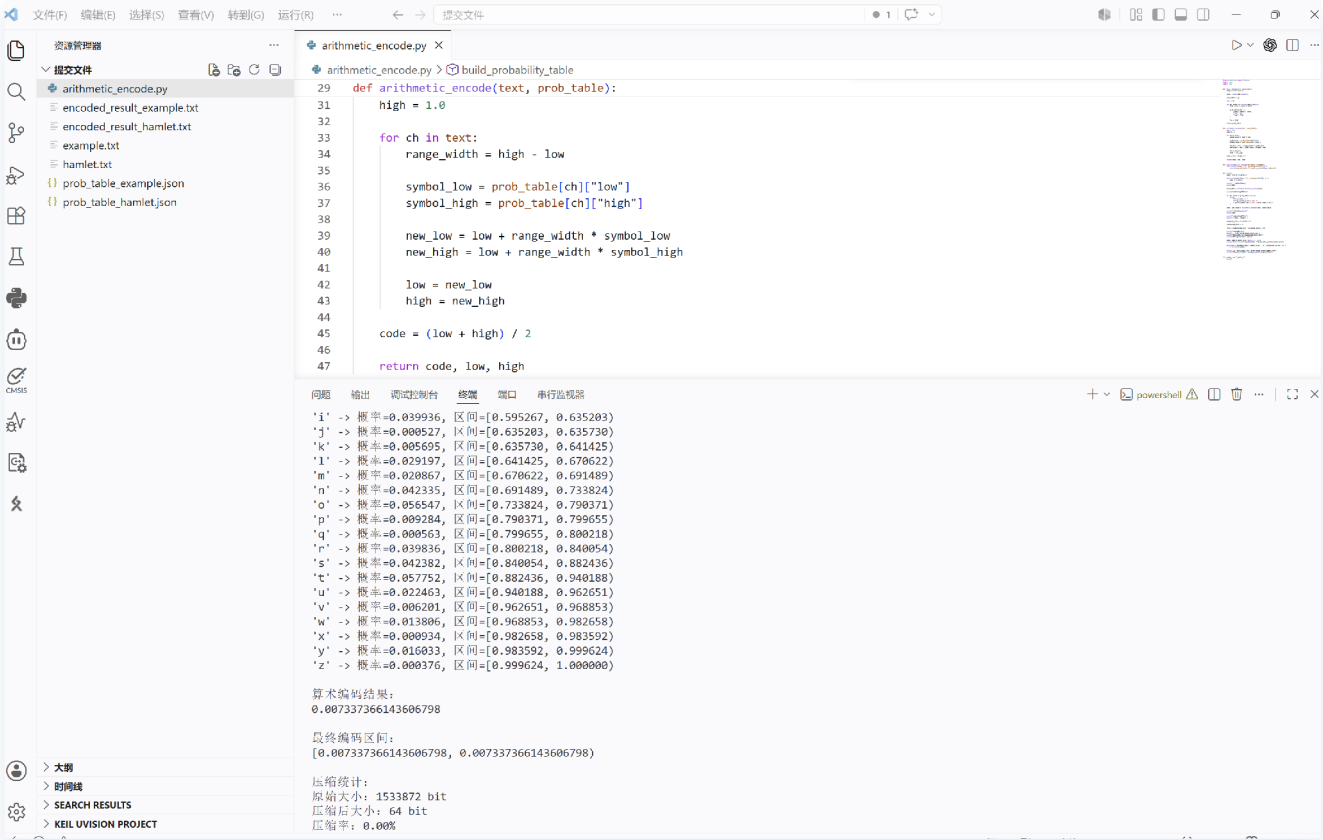


简要说明：

该截图展示了 `example.txt` 文本文件的算术编码实验结果。程序能够正确读取输入文本，并自动统计字符出现频率，根据字符概率划分对应的概率区间。随后程序利用算术编码算法不断缩小编码区间，最终输出对应的算术编码结果以及最终编码区间。

同时程序还生成了对应的概率区间表 JSON 文件与编码结果文件，控制台输出中还显示了原始文本大小、压缩后大小以及压缩率等统计信息，验证了程序实现的正确性。

(2) hamlet.txt 长文本编码结果与概率区间表截图



简要说明：

该截图展示了长文本文件 hamlet.txt 的算术编码实验结果。程序能够正确处理大规模文本，并自动生成对应的字符概率区间表与编码结果文件。由于长文本中字符分布更加稳定，因此字符概率统计更加准确，能够更好地体现算术编码对数据压缩的优势。

实验结果中可以观察到，空格、字母 e、字母 t 等高频字符对应较大的概率区间，而低频字符则对应较小区间，符合算术编码根据字符概率动态划分区间的基本思想。

同时控制台中输出了原始文件大小、压缩后大小以及压缩率等信息，进一步验证了程序对于不同规模文本均能够稳定运行。

3. 核心代码展示和分析

(1) 统计字符频率

```
freq = Counter(text)

total = sum(freq.values())
```

代码分析：

该部分代码用于统计文本中每个字符出现的频率。程序利用 Python 的 Counter 类快速完成字符计数，并计算字符总数，为后续概率计算与区间划分提供基础数据。字符频率越高，其对应概率区间越大。

(2) 构建字符概率区间

```
for ch, count in sorted(freq.items()):
    high = low + count / total

    prob_table[ch] = {
        "prob": count / total,
        "low": low,
        "high": high
    }

    low = high
```

代码分析：

该部分代码用于构建字符概率区间。程序按照字符顺序依次划分 $[0, 1)$ 区间，每个字符都会对应一个唯一子区间。字符出现概率越高，其对应区间长度越大。最终所有字符区间之和刚好覆盖整个 $[0, 1)$ 区间。

(3) 算术编码核心过程

```
for ch in text:
    range_width = high - low

    symbol_low = prob_table[ch]["low"]
    symbol_high = prob_table[ch]["high"]

    new_low = low + range_width * symbol_low
    new_high = low + range_width * symbol_high

    low = new_low
    high = new_high
```

代码分析：

该部分代码是算术编码算法的核心。程序不断根据当前字符对应的概率区间缩小编码范围。每输入一个字符，当前编码区间都会进一步收缩，最终形成一个极小区间，从而唯一表示整个字符串。

随着文本长度增加，编码区间会越来越小，因此能够实现较高的数据压缩效率。

(4) 生成最终编码值

```
code = (low + high) / 2
```

代码分析：

当所有字符处理完成后，程序取最终区间的中点作为最终算术编码结果。由于该数值位于最终区间内部，因此能够唯一表示整个输入字符串。

这种方式相比逐字符编码的方法更加紧凑，也是算术编码区别于霍夫曼编码的重要特点。

(5) 保存概率区间表

```
save_probability_table(  
    prob_table,  
    f"prob_table_{input_name}.json"  
)
```

代码分析：

程序利用 JSON 文件保存字符概率区间表，便于后续查看、分析与解码处理。由于不同输入文件会自动生成不同名称的 JSON 文件，因此程序能够同时支持多个文本实验。

(6) 保存编码结果

```
with open(  
    f"encoded_result_{input_file}",  
    "w",  
    encoding="utf-8"  
) as f:  
    f.write(str(code))
```

代码分析：

该部分代码用于保存最终算术编码结果。程序会根据输入文件名自动生成对应输出文件，便于后续查看编码结果，同时也提高了程序整体自动化程度。

(7) 压缩率统计

```
original_bits = len(text) * 8

compressed_bits = 64

ratio = compressed_bits / original_bits * 100
```

代码分析：

程序通过比较原始文本大小与压缩后编码大小计算压缩率。实验中原始文本按每字符 8 bit 统计，而最终编码结果采用浮点数表示，因此压缩后大小近似按照 64 bit 进行统计。

实验结果表明，对于规模较大的文本，算术编码能够有效减少数据存储空间。

(8) 实验结果分析

通过本次实验可以发现，算术编码能够有效实现文本数据压缩。对于 example.txt 等简单文本，程序已经能够正确完成概率统计、区间划分以及编码输出；而对于 hamlet.txt 等长文本，程序依旧能够稳定运行，并获得较好的压缩效果。

与霍夫曼编码不同，算术编码并不是为每个字符分配固定编码，而是通过不断缩小区间，将整个字符串映射为一个实数，因此在理论上能够更加接近信息熵极限。

本实验不仅加深了对算术编码原理的理解，也进一步熟悉了 Python 文件处理、概率统计、区间计算以及 JSON 数据处理等相关知识，为后续多媒体数据压缩算法学习奠定了基础。